

ENGINEERING

# Melting Moments

Technical Documentation

---

July 2026

Version 1.0

Melting Moments Catering & Guido's Gourmet, Victoria BC

# Contents

---

## 1 Overview

What the site does · Two brands, one codebase · Technology stack · Project structure · Scripts and tooling

---

## 2 Architecture

System overview · Rendering strategy · Data model · Backend conventions · Authentication and sessions · Admin console architecture · Security posture

---

## 3 Operations

Environments and topology · Environment variable reference · Deployment runbook · Domain cutover checklist · Content seeding · Backups · Troubleshooting

---

## 4 Search and Discovery

Metadata system · OpenGraph pipeline · Structured data graph · llms.txt · Sitemap and robots · Content guidelines

---

# Overview

---

The marketing and intake site for two Victoria, BC food brands: Melting Moments Catering, a full-service event caterer, and Guido's Gourmet, ready-made meals for delivery or pickup.

## What the site does

The application has four responsibilities:

- **Public marketing pages** for both brands: menus, service pages, testimonials, FAQ, and service area, with structured data for search on every page.
- **Lead intake.** Catering quote requests arrive through `/contact` (a JSON POST to `/api/contact`) and Guido's meal orders through `/guidos/order` (`/api/guidos-order`). Both endpoints save the lead to Convex first, then send an owner notification and a customer confirmation through Resend. Email failure never loses a lead.
- **An owner console** at `/admin`, protected by password, covering menus, Guido's products and orders, inquiries, FAQs, testimonials, site copy, the announcement banner, and settings.
- **Fast, resilient rendering.** Public pages are static with incremental static regeneration (a 300-second window on every public route). Admin edits do not wait for that window: server actions invalidate the content cache immediately, so changes appear on the public site on the next request.

Data lives in Convex. Public reads go through the cache layer in `src/lib/cms.ts`, and every public page carries hardcoded fallback copy, so a Convex outage degrades the content rather than failing the build or the request.

## Two brands, one codebase

Melting Moments Catering owns most routes. Guido's Gourmet lives under the `/guidos` subtree with its own menu, ordering flow, product catalog, and page titles. Requests arriving on a `guidosgourmet` host are permanently redirected by the middleware to `https://meltingmoments.ca/guidos` with the path preserved, so the secondary domain needs no separate deployment.

## Technology stack

| LAYER                          | CHOICE                                                                               | VERSION                 |
|--------------------------------|--------------------------------------------------------------------------------------|-------------------------|
| Framework                      | Next.js (App Router)                                                                 | 16.2.7                  |
| UI                             | React / React DOM                                                                    | 19.2.4                  |
| Language                       | TypeScript                                                                           | 5.x                     |
| Backend and database           | Convex                                                                               | 1.36.1                  |
| Transactional email            | Resend                                                                               | 6.12.2                  |
| Image processing (OG crops)    | sharp                                                                                | 0.34.5                  |
| Admin drag-and-drop reordering | @dnd-kit                                                                             | core 6.x, sortable 10.x |
| Analytics                      | @vercel/analytics                                                                    | 2.x                     |
| Styling                        | Hand-written CSS ( <code>globals.css</code> , <code>admin.css</code> ), no framework |                         |

Admin sessions are HMAC-SHA256 signed, expiring tokens rather than JWTs, verified in both the Node runtime ( `src/lib/auth.ts` ) and the proxy runtime ( `src/proxy.ts` ). Chapter 2 covers the full scheme.

### NEXT.JS VERSION NOTE

This project runs a Next.js release whose APIs differ from older documentation and most examples found online. Request middleware lives in `src/proxy.ts` , and cache invalidation uses `updateTag` . The authoritative reference is the documentation bundled with the installed release under `node_modules/next/dist/docs` . Read the relevant guide there before writing framework-facing code, and heed its deprecation notices.

## Project structure

| PATH                         | CONTENTS                                                                                                                                            |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>convex/</code>         | Convex backend: schema, queries and mutations per table, guarded seed functions. Deployed separately from the frontend.                             |
| <code>docs/</code>           | Team documentation and owner-facing PDFs.                                                                                                           |
| <code>public/</code>         | Static assets: photography, generated OG crops ( <code>public/og</code> ), favicons, Guido's product shots.                                         |
| <code>scripts/</code>        | <code>generate-og.mjs</code> , which regenerates the 1200x630 OpenGraph crops.                                                                      |
| <code>src/app/</code>        | App Router routes: public pages, the <code>/admin</code> console, <code>/api</code> routes, sitemap, robots, and the <code>llms.txt</code> handler. |
| <code>src/components/</code> | Shared public-site components: nav, footer, shell, announcement banner, page transitions.                                                           |
| <code>src/lib/</code>        | Auth (sessions), cms (cached Convex reads), seo (JSON-LD builders), sanitize (escaping helpers).                                                    |
| <code>src/proxy.ts</code>    | Request proxy: admin gating, session renewal, rate limiting, security headers, domain redirect.                                                     |

## Scripts and tooling

| COMMAND                                   | WHAT IT DOES                                                                  |
|-------------------------------------------|-------------------------------------------------------------------------------|
| <code>npm run dev</code>                  | Next.js dev server                                                            |
| <code>npm run build</code>                | Production build                                                              |
| <code>npm run start</code>                | Serve the production build                                                    |
| <code>npm run lint</code>                 | ESLint                                                                        |
| <code>npx convex dev</code>               | Sync <code>convex/</code> to the dev deployment on save                       |
| <code>npx convex deploy</code>            | Deploy <code>convex/</code> to the production deployment                      |
| <code>node scripts/generate-og.mjs</code> | Regenerate OpenGraph crops in <code>public/og/</code> from source photography |

Local development needs Node 20.9 or newer, npm, and a Convex account. Two terminals run side by side: `npx convex dev` provisions and syncs the dev Convex deployment (writing `CONVEX_DEPLOYMENT` and the deployment URL into `.env.local` ), and `npm run dev` serves the site at `localhost:3000` . To use the admin console locally, set `ADMIN_PASSWORD` in `.env.local` and set the identical value on the dev Convex deployment. The full environment variable reference is in Chapter 3.

### **DEPLOYS ARE TWO-PART**

The Convex backend deploys separately with `npx convex deploy`. Pushing to git or Vercel does not update Convex functions. Forgetting the backend half is the most common deployment mistake; the runbook in Chapter 3 covers the correct order.

# Architecture

System design as it exists in the repository: services, rendering, the data model, backend conventions, authentication, the admin console, and the security posture.

## System overview

The application is built on three managed services:

- **Vercel** hosts the Next.js App Router application. It serves the public marketing site as static pages with incremental static regeneration, renders the admin console as dynamic server-rendered routes, runs the middleware in `src/proxy.ts` on every request, and exposes two JSON API routes for the public forms. Vercel Analytics loads from the root layout.
- **Convex** is the system of record. All persistent data (catering menu, CMS copy, FAQs, testimonials, inquiries, Guido's products and orders, the activity log) lives in the tables defined in `convex/schema.ts`. The Next.js server talks to Convex over HTTP using `fetchQuery` and `fetchMutation` from `convex/nextjs`. Public pages never open a Convex websocket; the reactive client wraps only the admin tree.
- **Resend** sends transactional email: owner notification and customer confirmation for catering inquiries, and owner notification for Guido's orders. Email failures are logged and never fail the request, because the lead is already saved in Convex before any email is attempted.

## Request paths

**Public page:** browser, then the proxy middleware (security headers, rate limiting, host redirect), then the ISR cache, then the page render, which reads Convex through the cached getters in `src/lib/cms.ts`.

**Admin write:** browser, then a server action, then `requireAdmin()` (session cookie check), then `fetchMutation` carrying `adminSecret`, then the Convex mutation (`assertAdmin`, validation, the write, and the activity-log entry in the same transaction), then cache invalidation (`updateTag` plus `revalidatePath`), then a typed result object back to the client component.

## Rendering strategy

Public routes are static with ISR:

- Every public page, and the `/llms.txt` route handler, exports `export const revalidate = 300`.
- All public-page Convex reads go through `unstable_cache` in `src/lib/cms.ts` with `{ revalidate: 300, tags: ['cms'] }`. The getters are `getSettings`, `getTestimonials`,

`getMenuItems` , `getFaqs` , and `getGuidosProducts` .

- `getSettings` is outage-safe: a Convex failure returns `{}` and every page renders its hardcoded fallback copy instead of a 500. The other getters are wrapped in `.catch(() => [])` at their call sites.

Admin saves that touch public content invalidate the cache immediately:

- `updateTag('cms')` expires the CMS cache entries at once. The one-argument `revalidateTag` form is deprecated on this Next version; `updateTag` gives read-your-own-writes semantics.
- `revalidatePath('/', 'layout')` refreshes every public route.
- `revalidatePath` of the specific admin page refreshes the server-rendered admin data.

Content edits therefore appear on the public site as soon as the next request comes in. The 300-second window only matters for changes made outside the admin console, for example edits made directly in the Convex dashboard. Inquiry and Guido's-order actions hold no public-facing content, so they revalidate only `/admin` paths and skip the `cms` tag.

Admin routes are dynamic. `src/app/admin/layout.tsx` awaits `connection()` (this Next version's replacement for `unstable_noStore` ) and calls `requireAdminOrRedirect()` , which reads cookies. Nothing under `/admin` is ever cached.

Route-level redirects: `/quote` issues a 308 `permanentRedirect` to `/contact` (deliberately kept out of the sitemap), and `/gallery` permanently redirects to `/catering` via `next.config.ts` .

## Data model

All tables are defined in `convex/schema.ts` ; Convex adds `_id` and `_creationTime` to every document. A recurring convention: fields with a fixed set of values ( `status` , `brand` , `deliveryMethod` ) are stored as `v.string()` in the schema so existing rows always remain valid, and the allowed values are enforced at mutation arguments instead, via `v.union(v.literal(...))` or the enums in `convex/lib.ts` .



| TABLE            | PURPOSE                                                                                 | NOTABLE FIELDS AND RULES                                                                                                                                                                                                                                                                                |
|------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| menuItems        | Catering menu content for /menus and the admin Menu Editor. Index by_category .         | category (uppercase section name), name , description , optional numeric price (feeds the structured-data Offer), priceLabel (display string such as \$8.95/pp ), orderIndex , isActive , isFeatured . Only active items reach the public site.                                                         |
| inquiries        | Catering leads from /contact . Indexes by_status , by_submittedAt .                     | Customer fields( name , email , phone , eventType , guestCount , date , venue , notes ); status is one of new , contacted , booked , declined , archived . adminNotes is admin-editable; customer notes are immutable after create. archivedFromStatus supports restore.                                |
| businessSettings | Free-form key-value store, the CMS backbone. Index by_key .                             | Page copy overrides, banner state ( banner_* ), business identity( name , address , phone , social_* ), the notification preference emailOnNewInquiry , and menu_category_order . getAll is intentionally public-read; save is the atomic multi-key upsert behind every settings-style save.            |
| faqs             | CMS-managed FAQ entries.                                                                | question , answer , optional category ( catering or guidos ; undefined means general, grouped under Catering on the public page), orderIndex , isActive .                                                                                                                                               |
| testimonials     | CMS-managed reviews.                                                                    | author , optional role , text , optional rating (integer 1 to 5, enforced by normalizeRating ), optional brand (undefined means both), orderIndex , isActive . Public pages currently render all active testimonials without brand filtering; the field is stored but not yet used for display routing. |
| activityLog      | Append-only record of CMS changes, shown on the admin dashboard. Index by_performedAt . | action , section , optional details , performedAt . Retention is 90 days, enforced by bounded pruning on write.                                                                                                                                                                                         |
| guidosProducts   | Guido's Gourmet product catalog. Index by_category .                                    | name , category , priceFrom , optional sizes array of { label, price } , optional image path, isAvailable , isLimitedEdition , orderIndex . list is public-read; the public menu renders unavailable products as sold out rather than hiding them.                                                      |
| siteImages       | Dormant. Retained schema for the removed media module.                                  | storageId , title , alt , optional section , orderIndex , uploadedAt . No code reads or writes this table; it exists so any existing rows remain valid. Remove only after confirming the table is empty in both deployments.                                                                            |
| guidosOrders     | Guido's orders from /guidos/order . Indexes by_status , by_submittedAt .                | customerName , customerEmail , customerPhone , items (free-text summary), deliveryMethod ( delivery or pickup ), optional deliveryAddress and notes , status ( received , preparing , ready , delivered , picked_up ), submittedAt .                                                                    |

## Backend conventions

These rules hold across every module in `convex/` :

- **assertAdmin on everything non-public.** Every admin query and mutation takes an `adminSecret` argument and calls `assertAdmin` first. The check fails closed: if `ADMIN_PASSWORD` is unset on the Convex deployment, every call is rejected. The only public functions are the reads the public site needs, the `create` mutations behind the public forms, and a few non-PII `count` queries for the dashboard.
- **Partial-patch updates.** `update` mutations build a patch object containing only the fields the caller provided; omitted fields are untouched. Sentinel values handle deletion, for example `price: null` clears a menu item's price.
- **Append semantics for new rows.** When `orderIndex` is omitted on create, the mutation appends to the end of the relevant group. Moving a menu item to a different category without an explicit position also appends to the destination category.
- **Atomic reorders.** Each sortable table has a `reorder` mutation that receives the full ordered id array and patches `orderIndex` for every row inside a single Convex transaction. Any missing id, or an id belonging to a different category, throws and rolls the whole reorder back. There is no client-side sequencing of individual moves.
- **Activity logging inside the mutation.** `logActivity(ctx, entry)` inserts the log row in the calling mutation's transaction, so the log entry and the change it describes commit or fail together. The same call prunes at most 25 entries older than 90 days, keeping log writes bounded.
- **Guarded seeds.** `seed.ts` exposes `seedMenuItems` and `seedGuidosProducts`, run from the CLI. A seed refuses to touch a non-empty table and returns a skip report; passing `force: true` wipes the table and reinserts the hardcoded data. Chapter 3 covers usage.
- **Shared validators.** `requireText`, `assertPrice`, `validateSizes`, `normalizeRating`, `clampLimit`, and `truncateDetail` in `convex/lib.ts` keep validation identical across modules.

## Authentication and sessions

There is a single admin identity, authenticated by password.

**Token format.** The session token is `v2.<expiresAtMs>.<hmacHex>` where the HMAC is SHA-256 keyed by `ADMIN_PASSWORD` over `'melting-moments-admin-session.' + expiresAtMs`. The format is implemented twice, deliberately in lockstep: `src/lib/auth.ts` uses Node `crypto` for server actions, and `src/proxy.ts` uses WebCrypto because the middleware runtime has no Node `crypto`. Any change to the format must land in both files. Old v1 non-expiring tokens fail verification by design; holders log in again.

**Login.** The login action compares the submitted password against `ADMIN_PASSWORD` in constant time (both sides are SHA-256 hashed first so `timingSafeEqual` sees equal lengths), then mints a 7-day token and sets it as the `admin_token` cookie: `httpOnly`, `secure` in production, `sameSite: 'strict'`, `path: '/'`.

**Login rate limiting.** An in-memory, per-instance limiter applies exponential backoff per IP after 5 failures (30 seconds, doubling per additional failure, capped at 32 minutes), keyed by `x-forwarded-for`. A global bucket capped at 100 failures backstops distributed guessing across IPs; it is cleared on any successful login so accumulated bot noise can never lock out the real admin. Stale entries are pruned after an hour of inactivity.

**Middleware protection and sliding renewal.** For every `/admin` request (excluding `/admin-login`), `src/proxy.ts` verifies the cookie: format, expiry, and HMAC in constant time. Invalid or missing tokens redirect to `/admin-login`; a valid session visiting `/admin-login` redirects to `/admin`. When a valid token has less than 3.5 days remaining, the middleware mints a fresh 7-day token on the response with the same cookie attributes, so an active admin never gets logged out mid-week while an abandoned session still expires.

**Server actions.** Every mutating server action starts with `requireAdmin()`, which re-verifies the cookie and throws `AdminAuthError` on failure. Actions catch that error and return `{ success: false, error: 'unauthorized' }`; a missing `ADMIN_PASSWORD` is rethrown as a genuine server misconfiguration instead of being masked as an auth failure.

**Defense in depth.** `src/app/admin/layout.tsx` calls `requireAdminOrRedirect()` on every admin render, so even if a request bypassed the middleware, the layout bounces unauthenticated users to `/admin-login` before any data is fetched.

#### THE CONVEX LAYER VERIFIES INDEPENDENTLY

`ADMIN_PASSWORD` must be set to the identical value in both the Vercel environment and the Convex deployment environment. Next.js verifies the session; Convex verifies the `adminSecret` each call carries. A mismatch produces a working login whose every action fails with "Unauthorized".

## Admin console architecture

Each admin module follows the same shape:

- **Server component** `page.tsx` fetches the module's data with `fetchQuery` carrying `adminSecret`, serializes Convex documents into plain client-safe objects (ids as strings, timestamps as ISO strings), and renders the client component inside a `ToastProvider`.
- **Client component** owns the working copy in local state. Writes are optimistic: the UI updates first, the server action runs, and on failure the previous state is restored (deletes reinsert the row at its original index) and an error toast explains what happened.
- **Server action** `actions.ts` guards with `requireAdmin()`, validates input, calls the Convex mutation with `adminSecret`, and invalidates caches.

**Result convention.** Every action returns a discriminated union:

```
type ActionResult =
  | { success: true; id?: string }
  | { success: false; error: 'unauthorized' | 'invalid' | 'failed'; message?: string };
```

`unauthorized` comes from two places: `AdminAuthError` thrown by `requireAdmin()`, and Convex errors whose message contains "Unauthorized" (the secret-mismatch case). Clients treat `unauthorized` as session death: they roll back optimistic state and show the persistent `SessionExpiredToast`, whose link performs a full anchor navigation to `/admin-login` because the session is dead and a fresh server render is the safest path. `invalid` carries a user-facing validation message; `failed` is a generic server-side failure that has already been logged.

## Shared component kit

The kit lives in `src/app/admin/_components` and is re-exported through `index.ts`:

| EXPORT                                                                                                                                                                | PURPOSE                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ToastProvider</code> , <code>useToast</code>                                                                                                                    | Context-based toast stack (success, error, info) with auto-dismiss; errors stay visible longer.                                                                               |
| <code>ConfirmDialog</code>                                                                                                                                            | Modal confirmation; the destructive variant places initial focus on Cancel, and <code>busy</code> disables both buttons and Escape dismissal while an action runs.            |
| <code>SlidePanel</code>                                                                                                                                               | Right-hand edit drawer with a pinned footer for save and cancel; when <code>dirty</code> , closing asks for confirmation first.                                               |
| <code>SortableList</code> , <code>DragHandle</code>                                                                                                                   | dnd-kit wrapper for drag-to-reorder rows with keyboard support and screen reader announcements.                                                                               |
| <code>TextField</code> , <code>TextAreaField</code> , <code>SelectField</code> , <code>NumberField</code> , <code>ToggleSwitch</code> , <code>parseNumberField</code> | Labeled form controls with shared help-text and error wiring ( <code>aria-describedby</code> ).                                                                               |
| <code>EmptyState</code>                                                                                                                                               | Title, optional body, and an optional action for empty tables.                                                                                                                |
| <code>SkeletonRows</code>                                                                                                                                             | Loading skeleton with a deterministic width sequence (SSR and hydration safe).                                                                                                |
| <code>useDirtyGuard</code>                                                                                                                                            | Registers unsaved-changes state with the admin shell (sidebar navigation warns before leaving) and adds a <code>beforeunload</code> handler.                                  |
| <code>useAutoRefresh</code>                                                                                                                                           | Calls <code>router.refresh()</code> on an interval while the tab is visible, refreshes immediately on refocus, and returns a manual <code>refresh</code> for toolbar buttons. |

`modalBehavior.ts` is internal to the kit and not re-exported: it implements the shared modal stack so that a `ConfirmDialog` layered over a `SlidePanel` wins the Escape key, plus the focus trap and focus restore both components use.

## Security posture

**Headers.** `src/proxy.ts` sets all security headers on every matched response, and the matcher covers effectively all routes: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, HSTS for two years with `includeSubDomains; preload`, `X-XSS-Protection: 0` (the legacy auditor is deliberately disabled; CSP is the real defense), `Referrer-Policy: strict-origin-when-cross-origin`, a `Permissions-Policy` denying camera, microphone, and geolocation, and a CSP restricting scripts to self plus Vercel Analytics, styles and fonts to self plus Google Fonts, and connections to self, `*.convex.cloud` (https and wss), `*.resend.com`, and the Vercel analytics endpoints, with `frame-ancestors 'none'`, `base-uri 'self'`, and `form-action 'self'`. `'unsafe-eval'` is appended to `script-src` only outside production. `next.config.ts` intentionally sets no headers so there is a single source of truth.

**Input validation in layers.** Client-side checks give inline feedback; server actions and API routes enforce the authoritative limits: length caps on every field, email format via `isValidEmail`, enum checks, and an allowlist of writable settings keys in `SETTINGS_KEYS`. Convex mutations validate again through argument validators and the helpers in `convex/lib.ts`. The settings and banner modules share one validation module between the client and the action so the two layers cannot drift.

**Public form abuse controls.** Both `/api/contact` and `/api/guidos-order` carry a hidden honeypot field named `website`; a filled honeypot returns a fake success without persisting anything. The middleware applies a sliding-window rate limit of 10 POSTs per minute per IP to both endpoints (in-memory, per instance, with periodic pruning so the map cannot grow without bound). Login attempts have their own limiter, described above.

**Email hardening.** All user input is HTML-escaped with `escapeHtml` before interpolation into email bodies. Values placed in subject lines pass through `clean()`, which strips CR, LF, and tab characters and truncates to 120 characters, preventing header injection.

**CSV export hardening.** The inquiries CSV export quotes every cell, doubles embedded quotes, and prefixes any cell starting with `=`, `+`, `-`, or `@` with an apostrophe ( `sanitizeCsvCell` ) to neutralize spreadsheet formula injection through customer-controlled fields.

**JSON-LD escaping.** Structured data is serialized with `jsonLdSafe`, which escapes angle brackets and ampersands as unicode escapes so CMS-controlled strings cannot close the JSON-LD script element. Chapter 4 covers the details.

**Secrets.** `ADMIN_PASSWORD` serves three roles: the login password, the HMAC key for session tokens, and the shared secret Convex functions verify. It exists only in server environments (Vercel and the Convex deployment) and is passed server-to-server as `adminSecret`; it never reaches the browser. `RESEND_API_KEY` is server-only. The only `NEXT_PUBLIC_` variable is the Convex deployment URL, which is not a secret.

# Operations

How the site is deployed, configured, and kept healthy: environments, variables, the deployment runbook, domain cutover, seeding, backups, and troubleshooting.

## Environments and topology

| PIECE                        | VALUE                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------|
| Vercel project               | <code>melting-moments</code> (team <code>samuel-putras-projects-5ae50df2</code> )                    |
| Production alias             | <code>melting-moments-seven.vercel.app</code> (public; unique deployment URLs sit behind Vercel SSO) |
| Convex production deployment | <code>impartial-woodpecker-979</code>                                                                |
| Convex dev deployment        | <code>impressive-rhinoceros-224</code>                                                               |
| Production domain (target)   | <code>meltingmoments.ca</code> , DNS not yet cut over to Vercel                                      |
| Secondary domain (target)    | <code>guidosgourmet.ca</code> , redirects into <code>/guidos</code> once attached                    |

### WRONG-PROJECT TRAP

`melting-moments.vercel.app` (without `-seven` ) belongs to a different project. Do not use it.

There are two Convex deployments, and the Vercel environment decides which one a build talks to:

- Vercel Production env: `NEXT_PUBLIC_CONVEX_URL` points at the prod deployment.
- Vercel Preview env: `NEXT_PUBLIC_CONVEX_URL` points at the dev deployment, so preview branches never touch production data.
- Local dev: `.env.local` , written by `npx convex dev` , points at the dev deployment.

`guidosgourmet.ca` needs no separate site: `src/proxy.ts` 301-redirects any request whose host contains `guidosgourmet` to `https://meltingmoments.ca/guidos` , preserving the path.

## Environment variable reference

| VARIABLE               | VERCEL                                                | CONVEX ENV           | .ENV.LOCAL                                        | PURPOSE AND FAILURE MODE                                                                                                                                 |
|------------------------|-------------------------------------------------------|----------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| NEXT_PUBLIC_CONVEX_URL | Yes; Production points at prod Convex, Preview at dev | No                   | Yes                                               | Convex deployment URL for server reads and the admin client provider. Missing: public pages serve hardcoded fallback copy; admin pages cannot load data. |
| ADMIN_PASSWORD         | Yes                                                   | Yes, identical value | Yes, for local admin work (dev Convex must match) | Admin auth on both sides. Missing anywhere: admin fails closed. Mismatched: login works but every admin action returns "Unauthorized".                   |
| RESEND_API_KEY         | Yes                                                   | No                   | Optional                                          | Enables transactional email. Missing: submissions still save to Convex, but no emails send.                                                              |
| RESEND_FROM            | Yes                                                   | No                   | Optional                                          | Sender address; must be on a Resend-verified domain, otherwise falls back to <code>onboarding@resend.dev</code> , which cannot deliver to customers.     |
| OWNER_EMAIL            | Yes                                                   | No                   | Optional                                          | Recipient for new-inquiry and new-order notifications; defaults to <code>info@meltingmoments.ca</code> .                                                 |
| CONVEX_DEPLOYMENT      | No                                                    | No                   | Yes, written by <code>npx convex dev</code>       | Tells the Convex CLI which deployment to target. Not read by the application.                                                                            |

### ADMIN\_PASSWORD lives in two places

The same secret does two independent jobs:

1. On the Next.js side (Vercel env), it is the HMAC key for admin session tokens, and the login action compares the submitted password against it.
2. On the Convex side (deployment env), every admin query and mutation calls `assertAdmin`, which compares the `adminSecret` argument the Next server passes along against the deployment's own `ADMIN_PASSWORD`. If the deployment variable is unset, every admin call is rejected.

Set both, with identical values:

```
# Vercel (Production env), then redeploy for it to take effect
npx vercel env add ADMIN_PASSWORD production

# Convex production deployment (takes effect immediately)
npx convex env set ADMIN_PASSWORD <same value> --prod
```

For local work: put the value in `.env.local` and run `npx convex env set ADMIN_PASSWORD <value>` without `--prod`, targeting the dev deployment.

## Deployment runbook

Frontend and backend deploy separately, and forgetting the backend half is the most common mistake.

**Frontend (Vercel).** Push to `main`. Vercel builds and promotes to production automatically; other branches get preview deployments wired to the dev Convex deployment.

**Backend (Convex).** Run `npx convex deploy`. This pushes everything in `convex/` (schema, functions) to the production deployment. During development, `npx convex dev` continuously syncs the same files to the dev deployment; nothing reaches prod until you run `deploy`.

## Schema changes: order of operations

Convex validates every existing document against the schema at deploy time, and a deploy that existing rows violate fails. The codebase convention is to keep row-compatible types in the schema (status fields stay `v.string()`) and enforce enums at mutation args.

1. Write the schema change so existing rows remain valid: new fields optional, types widened rather than narrowed.
2. Deploy Convex first ( `npx convex deploy` ) so new functions and fields exist before any frontend code calls them.
3. Push the frontend change to `main`.
4. Backfill data through a mutation if needed; only then consider tightening the schema, and only after every row conforms.

## Post-deploy verification

1. Load the production alias, confirm real content renders (not fallback copy).
2. Log in at `/admin-login`, make a trivial content edit, confirm it appears on the public page immediately.
3. Revert the edit.

## Domain cutover checklist

DNS for `meltingmoments.ca` currently points at the legacy host, not Vercel. The cutover is the go-live switch.



## meltingmoments.ca

1. Vercel dashboard: project Settings, Domains, add `meltingmoments.ca` and `www.meltingmoments.ca`.
2. At the registrar, set the DNS records Vercel specifies (A or ALIAS for the apex, CNAME for `www`).
3. Wait for Vercel to verify the domain and issue certificates; confirm `https://meltingmoments.ca` serves the site.
4. Spot-check: `/admin-login` loads over the new domain, `/quote` 308-redirects to `/contact`, `/gallery` 301-redirects to `/catering`.
5. Resend dashboard: add and verify `meltingmoments.ca` (publish the DKIM and SPF records Resend provides at the registrar).
6. Vercel env: set `RESEND_FROM` to an address on the verified domain, for example `inquiries@meltingmoments.ca`, and confirm `RESEND_API_KEY` is set. Redeploy.
7. Submit a test inquiry and a test Guido's order; confirm the owner notification and customer confirmation both arrive and the leads appear in `/admin`.

The Resend DNS records can be published at any time; they do not conflict with the A/CNAME cutover. There is simply no point sending from a domain whose site is still elsewhere.

## guidosgourmet.ca

1. Add `guidosgourmet.ca` (and `www`) to the same Vercel project.
2. Update DNS at the registrar as above.
3. Verify `https://guidosgourmet.ca/menu` 301-redirects to `https://meltingmoments.ca/guidos/menu`, handled path-preservingly by `src/proxy.ts`.

No Resend setup is needed for this domain; all email sends from the `meltingmoments.ca` identity.

## Content seeding

`convex/seed.ts` exposes two guarded mutations, `seedMenuItems` and `seedGuidosProducts`. Guards:

- Both require an `adminSecret` argument matching the deployment's `ADMIN_PASSWORD`.
- Both refuse to touch a non-empty table unless `force: true` is passed.
- `force: true` wipes the table before reinserting the hardcoded data, destroying any owner edits. Treat it as dev-only.

Run against the dev deployment (the CLI targets `CONVEX_DEPLOYMENT` from `.env.local` by default):

```
npx convex run seed:seedMenuItems '{"adminSecret":"<dev ADMIN_PASSWORD>"}'
npx convex run seed:seedGuidosProducts '{"adminSecret":"<dev ADMIN_PASSWORD>"}'
```

## PRODUCTION SEEDS

Adding `--prod` targets production. Only do that for a first-time seed of empty tables; never pass `force: true` against prod without a fresh export in hand.

FAQs have no CLI seed. The first FAQ created through `/admin/faq` on an empty table auto-imports the built-in starter FAQs before inserting the new one. This keeps the public `/faq` page whole, because any CMS row replaces all of the hardcoded fallback questions.

## Backup and data notes

- **Export.** Convex dashboard, Data, export snapshot; or `npx convex export --path backup.zip` (add `--prod` for production). Take an export before any schema deploy or forced reseed.
- **Tables.** `menuItems`, `inquiries`, `businessSettings`, `faqs`, `testimonials`, `activityLog`, `guidosProducts`, `siteImages`, `guidosOrders`, all in `convex/schema.ts`.
- **siteImages is dormant.** The media module was removed; the table is retained only so existing rows stay valid.
- **activityLog self-prunes.** Each write deletes a bounded batch of 25 entries older than 90 days. Do not expect it to be a permanent record.
- **Customer leads matter most.** `inquiries` and `guidosOrders` exist only in Convex plus whatever email copies Resend delivered. They are the data that matters most in an export.

## Troubleshooting

| SYMPTOM                                                              | LIKELY CAUSE                                                                                                                                                                                                                         | FIX                                                                                                                                        |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Admin login rejects the correct password                             | <code>ADMIN_PASSWORD</code> unset in the Vercel env, or the login rate limiter is in backoff (5 failures per IP triggers exponential lockout, 30 s doubling up to 32 min)                                                            | Set the variable and redeploy; or wait out the lockout (it also resets on instance cold start)                                             |
| Login succeeds but every admin page or action returns "Unauthorized" | <code>ADMIN_PASSWORD</code> differs between Vercel and the Convex deployment env, or is unset on Convex ( <code>assertAdmin</code> fails closed)                                                                                     | <code>npx convex env set ADMIN_PASSWORD &lt;same value&gt; --prod</code> ; effective immediately, no redeploy needed                       |
| Everyone was logged out at once                                      | Expected after rotating <code>ADMIN_PASSWORD</code> : it is the HMAC key, so all existing session tokens fail verification                                                                                                           | Log in again                                                                                                                               |
| Forms save but no emails arrive                                      | <code>RESEND_API_KEY</code> unset; or <code>RESEND_FROM</code> unset or not on a Resend-verified domain; or the owner notification was turned off via the admin "Email on new inquiry" setting (suppresses the owner email only)     | Set both variables, verify the domain in Resend, check the toggle, then check Vercel function logs for <code>[Contact API]</code> errors   |
| Content edits do not appear on the public site                       | Editing a different deployment than you are viewing: Preview deployments and local dev point at dev Convex, production points at prod. Cache is not the issue; admin actions invalidate immediately on the deployment where they ran | Confirm which site you are viewing and which <code>NEXT_PUBLIC_CONVEX_URL</code> it was built with; worst case ISR re-fetches within 300 s |
| Public site shows placeholder or outdated fallback copy everywhere   | <code>NEXT_PUBLIC_CONVEX_URL</code> missing or empty in the Vercel env (public pages degrade to hardcoded fallbacks instead of failing), or the Convex tables are empty                                                              | Set the variable, redeploy, and seed or enter content                                                                                      |
| Local build fails with module or type errors that make no sense      | Stale incremental state in <code>.next</code> (and sometimes <code>tsconfig.tsbuildinfo</code> ) after dependency or config changes                                                                                                  | <code>rm -rf .next</code> and rebuild                                                                                                      |

# Search and Discovery

How metadata, OpenGraph images, structured data, and crawler-facing routes work, and the rules content editors need to follow.

## Metadata system

The root layout ( `src/app/layout.tsx` ) declares the site-wide defaults:

- **Title template.** The default title is `Melting Moments | Catering & Ready-Made Meals, Victoria BC` ; the template is `%s | Melting Moments Catering Victoria BC` . Pages set only their unique phrase, for example `title: 'Wedding Catering'` , and the template appends the brand and geo suffix. Guido's Gourmet routes carry their own brand and opt out with `title: { absolute: "Guido's Gourmet | ..." }` .
- **Canonical strategy.** `metadataBase` is `https://meltingmoments.ca` and the root layout sets `alternates: { canonical: './' }` . A relative `'./'` resolves against each route's own path, so every page canonicalizes to itself from a single declaration, with no per-page canonical boilerplate. The same `'./'` is used for `openGraph.url` , so social shares of deep pages do not all collapse onto the homepage URL.
- **Viewport.** `themeColor: '#070707'` , matching the dark brand background.
- **Other defaults.** `twitter: { card: 'summary_large_image' }` , `openGraph.locale: 'en_CA'` , a favicon set (32 and 16 pixel PNGs plus an Apple touch icon), and a default OG image.

The admin layout sets `robots: { index: false, follow: false }` so admin pages carry a noindex meta tag on top of the robots.txt disallow.

## OpenGraph pipeline

Every OG image is a generated 1200x630 JPEG crop in `public/og/` , produced from the source photography in `public/` by `scripts/generate-og.mjs` . Regenerate with `node scripts/generate-og.mjs` whenever a source photo changes.

The script uses sharp with `fit: 'cover'` and `position: 'attention'` , which centers the crop on the most visually salient region (this matters for the portrait and square sources), and writes JPEG at quality 80 with mozjpeg. The mapping lives in the `JOB$` array:

| SOURCE ( PUBLIC/ )      | OUTPUT<br>( PUBLIC/OG/ ) | USED BY                                                            |
|-------------------------|--------------------------|--------------------------------------------------------------------|
| hero-main.webp          | og-default.jpg           | Root layout default, /contact , /catering , and all /guidos routes |
| catering_menu_hero.webp | og-corporate.jpg         | /corporate                                                         |
| catering_menu_hero.webp | og-menus.jpg             | /menus                                                             |
| wedding_entree.webp     | og-weddings.jpg          | /weddings                                                          |
| private_dinner.webp     | og-private-events.jpg    | /private-events                                                    |
| copper_pots.webp        | og-about.jpg             | /about and /fountains                                              |
| macro_roulade.webp      | og-family-style.jpg      | /family-style                                                      |
| chef_plating_sauce.webp | og-chef.jpg              | /chef-paul                                                         |

There is no `og-guidos.jpg` yet because `public/guidos/` has no real hero photograph, only product shots and a placeholder SVG; the Guido's routes intentionally reuse `og-default.jpg` . When real Guido's photography lands, add a job entry and switch those routes over.

## Structured data graph

`src/lib/seo.ts` is the single source of truth for business identity and every JSON-LD builder. Pages must import from it rather than hardcoding the name, address, or phone.

- **SITE** holds the NAP (name, address, phone), email, coordinates, description, and Facebook URL. The postal code `V9A 6L2` must match `src/app/contact/page.tsx` and the email templates in `src/app/api/` .
- **BUSINESS\_ID** is `https://meltingmoments.ca/#business` , the canonical `@id` of the business entity. Every node links back to it, forming one connected graph rather than isolated islands.

The builders, and where each is emitted:

| BUILDER        | NODE TYPE                                                                                                                                                    | EMITTED BY                 |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| catererNode    | <code>['Caterer', 'FoodEstablishment']</code> with address, geo, areaServed, founder, hasMenu, and a makesOffer for Guido's                                  | Root layout, so every page |
| websiteNode    | WebSite linking back to the business                                                                                                                         | Root layout, so every page |
| breadcrumbList | BreadcrumbList                                                                                                                                               | Nearly every public page   |
| faqPage        | FAQPage with all displayed questions                                                                                                                         | /faq                       |
| menuNode       | Menu > MenuSection > MenuItem, with an Offer only for items that have a numeric price (display-only labels like \$8.95/pp get no Offer)                      | /menus                     |
| productNodes   | ItemList of Product nodes with AggregateOffer pricing (low and high computed from priceFrom and the sizes array, CAD, InStock / OutOfStock from isAvailable) | /guidos/menu               |
| personChefPaul | Person with worksFor pointing at BUSINESS_ID, reciprocating the business node's founder link                                                                 | /chef-paul                 |
| reviewNodes    | The business @id node carrying Review children                                                                                                               | /testimonials              |

Design decisions worth knowing:

- `Caterer` is not a core schema.org type, but answer-engine extractors use it widely; `FoodEstablishment` is a valid `LocalBusiness` subtype that satisfies Google's validators. The business node carries both.
- `reviewNodes` deliberately emits no `aggregateRating`: Google treats self-serving aggregate ratings on a business's own site as a structured-data policy violation.
- The root layout sources `sameAs` and opening hours from CMS settings ( `social_facebook`, `social_instagram`, `social_google_business`, `business_hours_note` ), so the owner can grow identity signals without a deploy. With no structured hours, the hours note renders as an `OpeningHoursSpecification` carrying only a `description`, the least-wrong way to express "by appointment".

## ESCAPING

The `JsonLd` server component renders the JSON-LD script element via `dangerouslySetInnerHTML`, serialized with `jsonLdSafe` from `src/lib/sanitize.ts`. `JSON.stringify` alone does not escape a closing script tag, so CMS-controlled content (an FAQ answer, a testimonial) could otherwise break out of the script element and execute as stored XSS. `jsonLdSafe` escapes angle brackets and ampersands as unicode escapes, which neutralizes the vector while remaining valid JSON-LD. `JsonLd` also adds `@context` when the node lacks one, so builders can be passed directly.

## llms.txt

`src/app/llms.txt/route.ts` serves a curated plain-text business summary for answer engines. It revalidates on the same 300-second ISR window as the pages and is composed from live CMS content:

- Business summary, website, and the five service lines with URLs.
- The full catering menu grouped by category with price labels, in the same order as `/menus` data.
- Available Guido's products with sizes, prices, and limited-edition flags (unavailable products are excluded).
- Service area, delivery terms, contact details, hours note, and booking terms.
- The complete FAQ list as question and answer pairs.

Each CMS getter degrades independently: if Convex is unreachable, the affected section falls back to a pointer at the live page instead of failing the whole response.

## Sitemap and robots

`src/app/sitemap.ts` lists all indexable routes with tiered priorities: conversion pages (home, `/catering`, `/contact`, `/menus`, `/weddings`, `/corporate`, `/guidos`, `/guidos/menu`) at 0.9 to 1.0, service pages at 0.8, utility pages at 0.6 to 0.7, legal pages at 0.3. `/menus` and `/guidos/menu` are marked `weekly` because their content is CMS-driven.

Two deliberate omissions:

- **No `lastModified`**. Stamping every URL with the build time is fake freshness that Google detects and then ignores; omitting the field entirely is better than an inaccurate one. Do not add it back unless it can carry real per-page modification times.
- **No `/quote`**. That route issues a permanent 308 redirect to `/contact` so link equity consolidates there; redirecting URLs do not belong in a sitemap.

`src/app/robots.ts` allows everything except `/admin`, `/admin-login`, and `/api/`, and points at `https://meltingmoments.ca/sitemap.xml`.

## Content guidelines

Rules for anyone editing copy, whether through the admin console or in code:

**Keep NAP consistent.** The business name, address, and phone must be identical everywhere they appear: `SITE` in `src/lib/seo.ts` (which feeds all structured data and `llms.txt`), the contact page, the email templates in `src/app/api/`, and external listings (Google Business Profile, Facebook). Inconsistent NAP dilutes local-search trust. If any element changes, update `SITE` first, then search the repository for the old value.

**FAQ categories map to public sections.** An FAQ's category is `catering` , `guidos` , or unset. The public `/faq` page renders two sections: "Catering" (category `catering` plus every uncategorized FAQ) and "Guido's Gourmet" (category `guidos` ). All displayed FAQs are emitted together in a single `FAQPage` node. The fallback set in `src/lib/fallback-faqs.ts` renders only when the CMS `faqs` table is empty or unreachable, and it is the same set the admin FAQ module's "Import starter FAQs" button seeds; edit that one file, never fork the copy.

**Menu category order is CMS-managed.** The section order on `/menus` comes from the `businessSettings` key `menu_category_order` , a comma-joined list of category names managed from the admin Menu Editor's "Order categories" panel. When unset, `DEFAULT_CATEGORY_ORDER` in `src/lib/menu-category-order.ts` applies. Categories present in the data but missing from the list are appended alphabetically, so a newly created category never disappears; it appears at the end until explicitly ordered. Because the setting is comma-joined, category names cannot contain commas (the admin action rejects them).

**Page titles.** New pages set a short unique phrase and let the template supply the suffix. Keep the phrase under roughly 30 characters so the combined title stays within display limits. Guido's pages use `title: { absolute: ... }` with the Guido's brand.

**Publishing latency.** Admin saves invalidate the cache immediately, so edits are live on the next request. The 300-second revalidation window only applies to changes made outside the admin console.